

计算机组成原理

Fall,2026

starAndHonor

1 序

本人对于计算机组成原理其实并没有特别深入理解,这里的笔记大多是结合 wgh 老师的 ppt 所写成,主要为考试所用,另外本专业的计组学时较少,没有学习 CPU 流水线等有关内容,故笔记中也不包含.

这是一份复习笔记,主要是为了快速考到及格,如果为了更高的分数,建议参考其他资料(老师的原 ppt,教材,作业,网课等等)进行复习.希望能够对大家有所帮助!

此外本笔记为初稿,可能存在较多细节错误,欢迎指正(提 issue)!

联系方式: <https://github.com/starAndHonor/noteforcs>

目录

1	序	2
2	发展历史	5
3	有符号整形的表示	5
3.1	三种表示方法	5
3.2	符号拓展	6
4	有符号加法和减法	6
5	Adder	6
5.1	1-bit full Adder	6
5.2	n bit ripple-carry adder	6
5.3	Carry-look-ahead Adder	7
5.4	减法的实现	8
5.5	对比两种加法器	8
6	无符号数的乘法	8
6.1	概念	8
6.2	阵列乘法	8
6.3	顺序乘法	9
7	booth algorithm	9
8	不恢复余数除法	10
9	IEEE 754	10
9.1	浮点数的表示	10
9.2	浮点数的运算	12
10	存储器	13
10.1	基本术语	13
10.2	SRAM 和 DRAM	14
10.3	使用小容量存储器构建大存储器	15
10.4	字扩展法	15
10.5	位拓展法	15
10.6	字位同时拓展法	15
11	内存的层级	15
11.1	buffer	15
11.2	时间局部性和空间局部性	15
11.3	四个基本问题	15
12	缓存映射策略	16
12.1	cache	16
12.2	读取数据	16
12.3	变量	16
12.4	Direct Mapping	17
12.5	Associative Mapping	17
12.6	组相连 Set Associative Mapping	18
12.7	valid bit	18
12.8	Replacement Algorithms	19
12.9	写策略	19
12.10	多级缓存	19
13	虚存	19
13.1	page	20

13.2	TLB(Translation Look-aside Buffer)转换检测缓冲区	20
13.3	缺点	20
13.4	选取 page 的大小	21
14	磁盘 Magnetic Hard Disk	21
14.1	组织形式	21
14.2	容量计算	21
14.3	延迟分析	21
15	指令集	21
15.1	两种指令形式	21
15.2	RISC 和 CISC	22
15.3	顺序执行和 branching	22
15.4	opcode 的设计	22
15.5	寻址方式	23
16	基本处理单元	23
16.1	多阶段	23
16.2	指令执行五步	23
16.3	硬件执行五步	24
16.4	控制信号	25
16.5	Generation of Control Signals	26
17	I/O 系统	26
17.1	访问 I/O 设备	26
17.2	I/O Device Interface	26
17.3	Program-controlled I/O(polling)	26
17.4	interrupt	26
17.5	DMA	27
17.6	三种方法比较	29

2 发展历史

- The Zero Generation: Mechanical Computers (1642–1945) — 第零代：机械计算机（1642–1945）
- The First Generation: Vacuum Tubes (1945–1955) — 第一代：电子管（1945–1955）
- The Second Generation: Transistors (1955–1965) — 第二代：晶体管（1955–1965）
- The Third Generation: Integrated Circuits (1965–1975) — 第三代：集成电路（1965–1975）
- The Fourth Generation: LSI & VLSI (1975–) 第四代：大规模/超大规模集成电路（LSI/VLSI, 1975–)

3 有符号整形的表示

3.1 三种表示方法

1. signed magnitude(原码)

1 位符号位	数值位:绝对值转化为 2 进制后的值			
1	0	1	0	1

表示 -5 .有 $+0$ 和 -0 ,且不能直接运算加减法.

2. 1's complement(反码)

1 位符号位	数值位正数为原码,负数绝对值原码取反			
1	0	1	0	1

表示 $-(\sim(0101)_2) = -(1010)_2 = -10$ 解决了加减法的问题(注意其独有的循环进位),但是仍然有 $+0$ 和 $-0(11\dots1)$ 的存在. 反码的范围 $-2^{d-1} + 1 \sim 2^{d-1} - 1$

反码	111...111	100...000	000...000	011...111
原码	100...000	111...111	000...000	011...111
十进制	$-2^{d-1} + 1$	-0	$+0$	$2^{d-1} - 1$

3. 2's complement(补码)

1 位符号位	数值位正数为原码,负数绝对值原码取反+1			
1	0	1	0	1

表示 $-(\sim(0101)_2 + 1) = -(1010 + 1)_2 = -1011_2 = -11$

解决的 $+0/-0$ 问题,规定 $10\dots0$ 表示 -2^{d-1} , 所以

反码		111...111	100...000	000...000	011...111
补码	100...000	111...111	000...000	011...111	
反码十进制		$-2^{d-1} + 1$	-0	$+0$	$2^{d-1} - 1$
十进制	-2^{d-1}	$-2^{d-1} + 1$	0	$2^{d-1} - 1$	

对于补码 x ,计算

$$-x = (\sim x) + 1$$

3.2 符号拓展

把一个有符号整数从 n 位扩展到 m 位 ($m > n$), 并且数值不变。对于补码或者反码来说

m-n 位数	1 位符号位	n-1 位数
00...00	0	...
11...11	1	...

也就是说反码或者补码的符号拓展是在前方增加符号位.

4 有符号加法和减法

算数溢出的判定: 算数溢出的判定的充分必要条件是如下:

相同符号相加,产生与他们不同符号的结果.

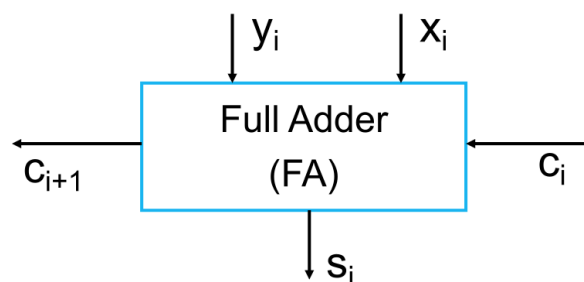
考虑将减法 $X - Y$ 变成 $X + (-Y)$ 计算, $-Y = \sim Y + 1$

5 Adder

5.1 1-bit full Adder

$$s_i = x_i \oplus y_i \oplus c_i$$

$$c_{i+1} = x_i y_i + x_i c_i + y_i c_i$$



其计算 c_i 的延迟是 $2T$, 计算 s_i 的延迟是 T

5.2 n bit ripple-carry adder

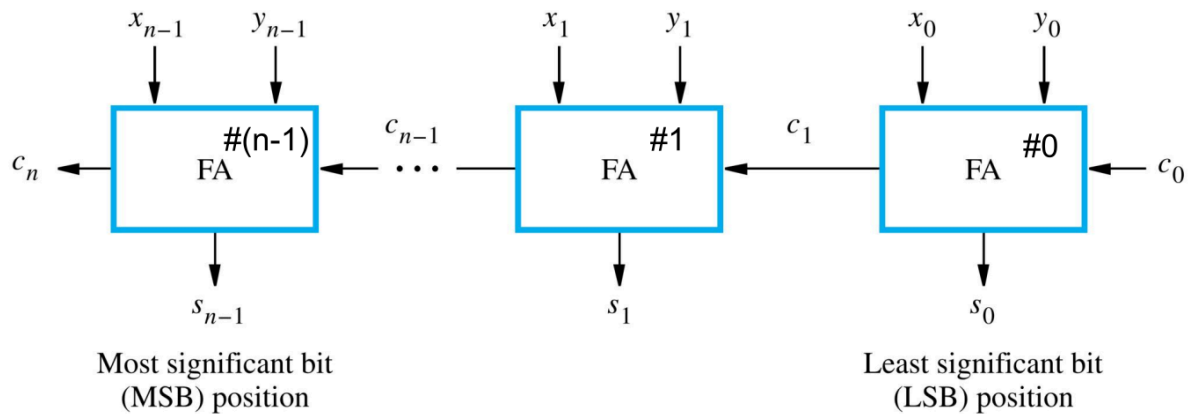
串联 n 个 full adder

$$\text{overflow} = x_{n-1}y_{n-1} \neg(s_{n-1}) + \neg(x_{n-1}) \neg(y_{n-1})s_{n-1}$$

上面这个式子是定义而来

$$\text{overflow} = c_n \oplus c_{n-1}$$

这个式子是硬件上的实现,二者等价



延迟分析: 计算 c_{n-1} 的延迟是 $2(n-1)T$, 计算 c_n 的延迟是 $2nT$, 计算 s_n 的延迟是 $(2n-1)T$

5.3 Carry-look-ahead Adder

$$s_i = x_i \oplus y_i \oplus c_i$$

$$c_{i+1} = x_i y_i + x_i c_i + y_i c_i = (x_i + y_i) c_i + x_i y_i$$

发现 c_{i+1} 只与 $x_i + y_i, x_i y_i$ 有关, 所以不妨令

$$G_i = x_i y_i$$

$$P_i = x_i + y_i$$

那么

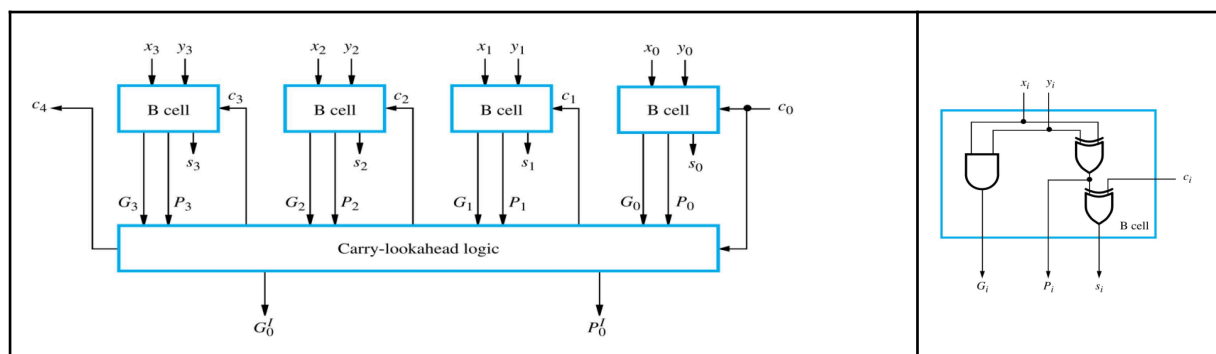
$$c_{i+1} = P_i c_i + G_i = P_i (P_{i-1} c_{i-1} + G_{i-1}) + G_i = \dots = P_i P_{i-1} \dots P_0 c_0 + P_{i-1} \dots P_1 G_0 + \dots + P_i G_{i-1} + G_i$$

注意到 $\oplus$ 运算和 $\vee$ 运算只有在 xy 为1的时候出现不同数值, 但是此时一定是进位, 此时 $G_i = x_i y_i$ 保证了此时一定进位, 所以可以将 P 定义成为 $x_i \oplus y_i$ 不改变左后的运算结果.

称为 $P = x_i \oplus y_i$ 为传播进位, 表示是因为低位进位而产生的当前位进位, 称为 $G = x_i y_i$ 为生成进位, 表示是哪怕前面没有进位也会产生进位的情况

由此带来新的理解, 把

$$c_{i+1} = \underbrace{G_i}_{\text{当前位的生成进位(不用传播)}} + \underbrace{P_i \dots P_{k+1} G_k}_{\text{中间的生成进位发生传播}} + \underbrace{P_i \dots P_1 P_0 c_0}_{\text{外部输入的初始进位发生传播}}$$

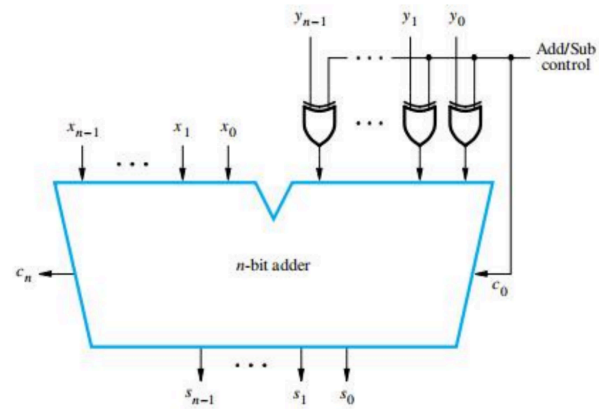


由此, 只需要使用硬件电路计算 P_i 和 G_i , 延迟为一个 T , 再使用与门加或门 $2T$ 计算得到 c_i , 然后复用 P_i 通过异或门异或 c_i 一个 T , 产生 s_i

所以其生成 c_i 的延迟是 $3T$,生成 s_i 的延迟是 $4T$

5.4 减法的实现

减法相当于 Y 取反加一后的加法,在之前的加法器 Y 的每一位前面加一层异或门,与 c_0 异或,加法的时候是 $c_0 = 0$,不改变输入,减法的时候 $c_0 = 1$,异或后获得 $\sim y$,然后刚好加上初始进位 c_0 ,变为 $\sim y + 1 = -y$ 实现了减法.



5.5 对比两种加法器

加法器	ripple	carry lookahead
优点	硬件电路简单	延迟较短且与位数无关
缺点	延迟与位数有关,相对较大	硬件电路复杂,扇入大延迟增高,功耗增大等

6 无符号数的乘法

6.1 概念

- Multiplicand 被乘数,一般用 M 表示
- Multiplier 乘数 一般用 Q 表示

6.2 阵列乘法

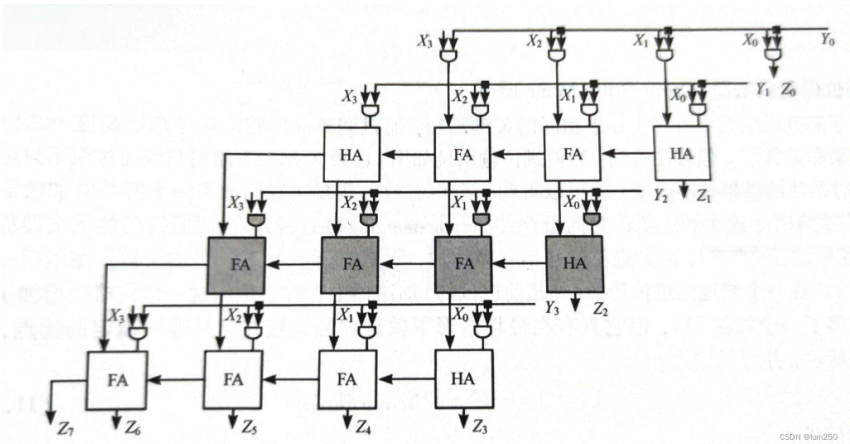


图 6 阵列乘法

模拟的竖式乘法,不考虑 HA 优化,即把上面的 HA 变为 FA,需要 n^2 个与门和 $n(n-1)$ 个 FA,因为第一行没有部分积不需要加法,直接输出 Y_0X 即可.

6.3 顺序乘法

$M = 1101$, $Q = 1011$, 计算 $P = M \times Q$

	M								
	1	1	0	1					
C	A				Q				
0	0	0	0	0	1	0	1	1	初始状态
0	1	1	0	1	1	0	1	1	Add
0	0	1	1	0	1	1	0	1	Shift
1	0	0	1	1	1	1	0	1	Add
0	1	0	0	1	1	1	1	0	Shift
0	1	0	0	1	1	1	1	0	No add
0	0	1	0	0	1	1	1	1	Shift
1	0	0	0	1	1	1	1	1	add
0	1	0	0	0	1	1	1	1	Shift

所以最后结果是 $P = 10001111_2$

7 booth algorithm

通过布斯算法可以直接对有符号数进行乘法计算。任何二进制数中连续的 1 可以被分解为两个二进制数之差：

$$\left(\overbrace{\dots 01\dots 10\dots}^n \right)_2 \equiv \left(\overbrace{\dots 01\dots 00\dots}^n \right)_2 - \left(\overbrace{\dots 00\dots 10\dots}^n \right)_2 \quad (7.1)$$

例如,对于0001111100我们将其分解为0010000000 - 0000000100 这样可以有效减少 partial sum 的计算

Bit i	Bit i-1	Version of Multiplicand Selected by bit i
0	0	$0 \times M$
0	1	$+1 \times M$
1	0	$-1 \times M$
1	1	$0 \times M$

这个表格本质上就是 式 (7.1) 的过程.即对于一串连续的一将其前面的第一个 0 设为 1,最后一个 1 设为-1,在连续 0 或一都设置为 0.

Multiplier 第 <i>i</i> 位	0	0	1	1	0	1	1	1	0
Booth 编码	0	1	0	-1	1	0	0	-1	0

上面表格的意思是 $001101110_2 = 000010000_2 - 000000010_2 + 010000000_2 - 000100000_2$ 在十进制下为 $110_{10} = 16_{10} - 4_{10} + 128_{10} - 32_{10}$ 计算过程中需要考虑部分积的符号拓展,拓展到 $n_A + n_B$ 位

8 不恢复余数除法

$A = 1000, B = 0011$.,计算 $P = A \div B$

C	M(除数)								
0	0	0	1	1					
C	A(余数)				Q(商)				
0	0	0	0	0	1	0	0	0	初始状态
0	0	0	0	1	0	0	0		Shift
1	1	1	1	0	0	0	0		Sub
1	1	1	1	0	0	0	0	0	set q
1	1	1	0	0	0	0	0		Shift
1	1	1	1	1	0	0	0		Add
1	1	1	1	1	0	0	0	0	set q
1	1	1	1	0	0	0	0		Shift
0	0	0	0	1	0	0	0		Add
0	0	0	0	1	0	0	0	1	set q
0	0	0	1	0	0	0	1		Shift
1	1	1	1	1	0	0	1		Sub
1	1	1	1	1	0	0	1	0	set q

- 需要在前面补充一个符号位C,因为我们需要观察符号位来判断q,然后通过q来决定加减.
- 恢复余数,此时余数是负数(C是 0),需要恢复,即加上M

	A(余数)				
	1	1	1	1	1
+	0	0	0	1	1
	0	0	0	1	0

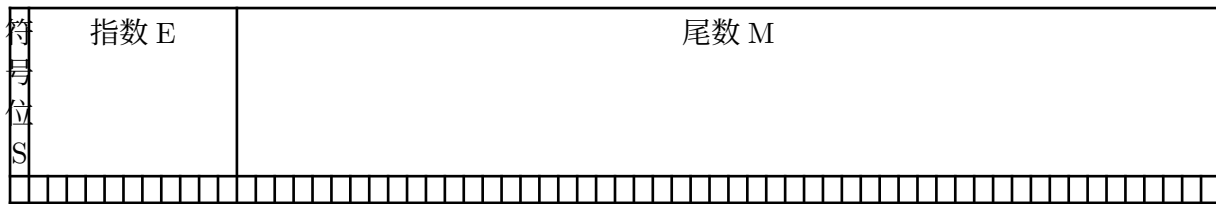
所以最终结果为商为0010,余数为0010

9 IEEE 754

9.1 浮点数的表示

假定一个浮点数可以写成 $(-1)^S \times 2^e \times M$

符号位 S	指数 E							尾数 M																							



9.1.1 指数的移码表示

指数 E 使用的是移码表示

$$E = e + \text{bias}$$

其中

$$\text{bias} = 2^{k-1} - 1$$

其中 k 是指数的位数.

其原因是为可以直接使用无符号方式比较,电路简单

此外,我们留出指数为全 0 和全 1 的情况用于表示特殊值.

- 指数全 0 表示非规格化数或者 0
- 指数全 1 表示无穷大或者 nan
- 指数在 $1 \sim 2^k - 2$ 表示规格化数

9.1.2 规格化浮点数

规格化的意思是把形式是 0.1... 的形式. 但是在 IEEE 中,规格化浮点数是以 1.xxxx 的形式表示的,这样可以省去第一个 1 的存储空间. 所以在存储的时候,尾数部分实际上存储的是小数点后面的部分.

$$x = (-1)^S \times 2^{e-\text{bias}} \times (1.M)$$

表示的最小绝对值为 $2^{-126} \times 1.0$

符号位 S	指数 E								尾数 M			
0/1	0	0	0	0	0	0	0	0	1	0	0...0	0

表示的最大绝对值为 $2^{127} \times (2 - 2^{-23})$

符号位 S	指数 E								尾数 M			
0/1	1	1	1	1	1	1	1	0	1	1...1	1	1

9.1.3 正负 0

E 全为 0, M 全为 0, S=0/1

9.1.4 正负无穷大

E 全为 1, M 全为 0, S=0/1

9.1.5 NAN

E 全为 1, $M \neq 0$, S=0/1

9.1.6 非规格化数

E 全为 0, $M \neq 0$, S=0/1 表示靠近 0 的数字(太小以至于规格化数无法表示),默认指数为 $1 - \text{bias}$, M 前面不再隐含 1,而是表示 0.M,称为渐进下溢

$$x = \pm 0.M \times 2^{1-\text{bias}}$$

9.2 浮点数的运算

9.2.1 对齐

对齐的方法是把指数小的转化成较大指数下表示. 例如 1.01×2^{-5} 和 1.0101×2^{-2} , 对齐就是将 1.01×2^{-5} 变为 0.00101×2^{-2}

9.2.2 加减

1. 对齐
2. 加减底数
3. 规格化

例如

$$1.0 - 0.875$$

转化为二进制为 $1 \times 2^0 - 1.11 \times 2^{-1}$

1. 对齐

$$1.11 \times 2^{-1} \rightarrow 0.111 \times 2^0$$

2. 加减底数

	1	.	0	0	0
-	0	.	1	1	1
	0	.	0	0	1

结果为 0.001×2^0

3. 规格化

1.0×2^{-3} , 注意是在对齐后的指数上减法, 不一定是在 0 上减法

9.2.3 乘除

1. 计算指数

乘法: $E = E_1 + E_2 - \text{bias}$ 除法: $E = E_1 - E_2 + \text{bias}$

2. 乘除运算底数, 并确定符号
3. 规格化

9.2.4 guard bits

在浮点数运算的时候应该多保留几位, 这样结果更加精确, 多保留的位数叫作保护位

9.2.5 三种截取方法

9.2.5.1 chopping

直接丢掉保护位, 优点是实现简单, 缺点是有偏, 始终变小

9.2.5.2 Von Neumann rounding(恒置 1 法)

如果要丢掉的部分全 0, 则 chopping, 如果要丢掉的部分有 1, 则将要保留的最后一位变成 1.

优点是无偏的, 实现不是特别难, 缺点是绝对误差大 $-1 \sim +1$

9.2.5.3 Rounding(0 舍 1 入法, 默认是这个)

如果要求掉的部分的最高位是 0, 那么就 chopping

如果要求掉的部分的最高位是 1 但是后面不是全 0,那么就要保留的数的最低位加上 1

如果要求掉的部分的最高位是 1 但是后面全 0,保留位最低位是 0 的话就 chop,如果是 1 的话就加 1(保证最低位是 0)

优点是绝对误差小,无偏,缺点是实现比较复杂

$0.xxx0000 \sim 0.xxx0111$ 变为 $0.xxx$

$0.xxx_{-1}1000$

- $x_{-1} = 0$ 那么就 $0.xxx$
- $x_{-1} = 1$ 那么就 $0.xxx + 0.001$

$0.xxx_{-1}1001 \sim 0.xxx1111$ 变为 $0.xxx + 0.001$

重点是需要转换和计算的时候多写几位然后截取

9.2.6 上溢和下溢

指数上溢,就是比可以表示的最大指数(127)大,可能产生 nan 或者 inf

指数下溢,就是比可以表示的最小指数(-126)小,可能变 0

尾数上溢,产生了进位,可以移动加上变指数解决

尾数下溢,是在对齐的时候右移导致的丢失,可以通过保护位和截取解决

尾数部分的溢出不是真的溢出,可以通过移位解决,指数部分才是真的溢出.

10 存储器

10.1 基本术语

10.1.1 字长和字数

- Word 一组固定大小的位被称为一个 word
- word length 一个 word 中位的数目

主存储器主要通过字长 $\times$ 字数来表示,例如 $4K \times 8$ 表示的是有字长是 8,字数是 4k 的存储器

内存一般一次传输和数据线数目一样的位数,外存一般按照 block 传输,一般远大于字长

10.1.2 编址方式

- memory address: 一个用于确定内存位置的数,地址的位数为 k ,则可以寻址的地址数目为 2^k 个
- big-endian: 大端序,高位低地址

0	1	2
3	4	5

- little-endian: 小端序,低位高地址

2	1	0
5	4	3

10.1.3 三种访存方式

- 顺序访存:访问位置 k ,经过 $1 \sim k-1$ 再到 k ,访问时间差别极大,例如磁带
- 随机访存:可以随机到某一个位置读取,访问时间基本不变,如半导体 ram

- 直接访存:先跳到某一个大致区域,然后在某一个区域具体查找,访问时间可变,如磁盘

10.1.4 磁盘性能评估

- Memory Access Time: 找到地址并获取数据的时间
- Memory Cycle Time: 找到地址并获取数据,然后准备好下一个地址的时间

可以知道访问时间比周期时间要短,因为周期还要考虑恢复时间.

- Transfer Rate: 单位时间写入或者读出数据的速度

10.1.5 Volatile/Nonvolatile

Volatile 断电数据就没了,比如说 RAM Nonvolatile 断电数据还在,比如说 ROM,磁盘,磁带,flash

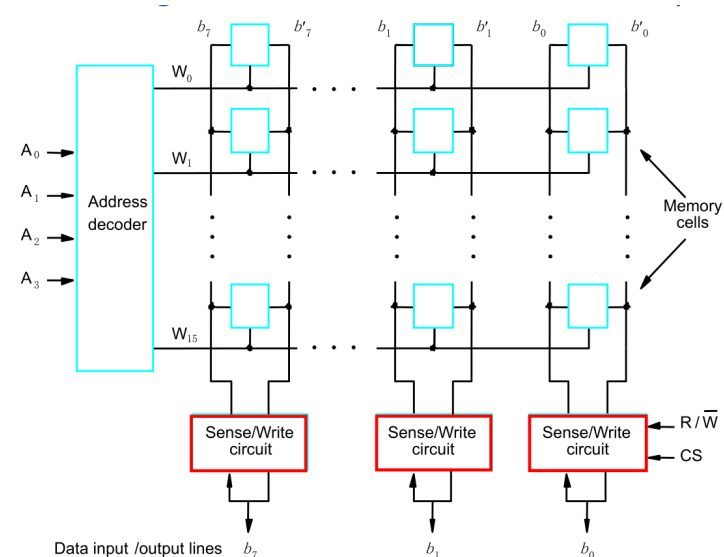
10.1.6 Erasable/Non-erasable

Erasable : 数据可以擦除并重新写入 /Non-erasable: 不可以重新写入

10.2 SRAM 和 DRAM

10.2.1 SRAM

单个硬件结构上是六个晶体管



地址线用于寻址,要过一个译码器所以会有一个 $\log$, 数据线用于写入所以与字长一样,可以理解为一次就拿出一行的数据操作. 假定大小为 $8K \times 16$, 那么需要 $\log_2 8K = 13$ 条地址线, 每一行有 16 位, 所以需要 16 条数据线.

10.2.2 DRAM

DRAM 是一个电容加一个晶体管的结构, 由于电容信号弱小, 所以访问需要经过放大器, 而且电容会漏电, 还需要定期刷新.

DRAM 的访问是二维的, 行地址和列地址分别经过一个译码器后选中一个电容进行读写. 假设是一个 $32M \times 8$ 的电容, 满足 $16384 \text{ rows} \times 2048 \text{ bytes}$ 的电容阵列, 那么需要 14 条行地址线和 11 条列地址线, 8 条数据线. 总地址线的数目应该等于 $14 + 11 = 25 = \log_2 32M$ 条.

fast page mode: 先选中一个行, 然后放入一个 latch (锁存器), 这样访问同一行的列就只需要列地址访问锁存器就行

10.2.3 延迟(latency)和带宽(bandwidth)

- 延迟: 开始传输第一个数据所需要的时间
- 带宽: 一秒钟内能够传输的比特数或字节数,取决于访问内存的速度和并行能力
- 有效带宽: $\text{bus width} \times \text{bus speed}$,还取决与数据线的水平

10.2.4 比较

	SRAM	DRAM
优点	• 速度快	• 高密度,低价格
缺点	• 低密度,高价格	• 速度慢 • 漏电问题,需要定期刷新

10.3 使用小容量存储器构建大存储器

10.4 字扩展法

有 $a \times c$ 容量的存储器,我们需要构造 $ab \times c$ 容量的存储器 那么就会产生 b 个行,然后我们要从这 b 行,字长为 a 的存储器中选取 1 个,在从这个存储器中选出一行,所以需要的线的数目为 $\log_2(ab)$,其中 $\log_2 a$ 个用于片内选择,剩下 $\log_2 b$ 个用于选择芯片在哪一行,需要 c 根数据线

10.5 位拓展法

有 $a \times c$ 容量的存储器,我们需要构造 $a \times bc$ 容量的存储器 那么就会产生 b 个列,使用 $\log_2 a$ 根地址线选择片内,使用 bc 根数据线

10.6 字位同时拓展法

目标是 $a \times b$,已有是 $c \times d$, 计算出矩阵排布为

$$R \times C = \frac{a}{c} \times \frac{b}{d}$$

需要地址线 $\log_2 a$ 条,其中 $\log_2 c$ 条用于片内选址, $\log_2 \frac{a}{c}$ 用于选取正确的行, b 根数据线用于传输数据

11 内存的层级

11.1 buffer

选取大而慢的设备的子集到一个小而快的设备,第 k 层小而快的设备充当第 $k+1$ 层大而慢的子集

最终所有数据都会存在最大最慢的设备上,数据在相邻的两层之间复制

Hit 表示在 k 层找到了, Miss 表示在 k 层没找到不得不去 $k+1$ 层找

11.2 时间局部性和空间局部性

时间局部性: 最近访问的在不久的将来还会被访问 → 将当前访问的复制到上层

空间局部性: 一个位置被访问,其周围也有很大概率被访问 → 将当前访问的周围复制到上层

11.3 四个基本问题

- Block Placement:如何放到上层
- Block Identification:如何在上层查找
- Block Replacement:在 miss 的时候如何替换

- Write Policy:如何执行写操作

12 缓存映射策略

12.1 cache

是放在 CPU 和 main memory 之间的 buffer,小而快的 SRAM,是硬件自动管理的,

0	a block
...	
k-1	

磁盘是按照 block 组织的,一个 block 是磁盘中 k 个连续的 word,一次复制到缓存中的一个 cache line 的内容是一整个 block.

tag	block		
	0	...	k-1

组成了一个 cache line,tag 用于存储标志信息,block 用于存储数据,cache 由很多 cache line 组成

12.2 读取数据

Hit:返回请求的数据

Miss:

1. 从主存复制到 cache,然后从 cache 返回
2. 从主存复制到 cache 的同时从 main memory 返回

12.3 变量

1. 一个 block 中有 W 个 word(注意不是字节)
2. 内存中有 B 个 block
3. 缓存中有 L 个 cache line

那么一个 block 中找到一个 word 需要

$$w = \log_2 W$$

那么一个内存中找到一个 block 需要

$$b = \log_2 B$$

那么一个缓存中找到一个 cache line 需要

$$l = \log_2 L$$

主存有 $w + b$ 个地址

12.4 Direct Mapping

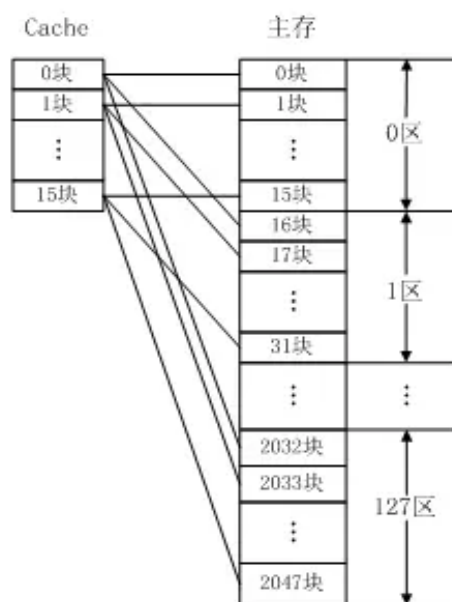


图 3-14 直接映射示意图

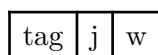
映射方式是第 i 个 block 到第 j 个 cache line

$$j = i \% L$$

其 tag 是

$$\text{tag} = \frac{i}{L}$$

主存地址结构



那么 j 有 $\log_2 L$ 位, tag 至少有 $\log_2 \frac{B}{L}$, w 为 $\log_2 W$

优点: 简单易实现, 便宜

缺点: 固定位置, 可能出现高 miss 的情况

12.5 Associative Mapping

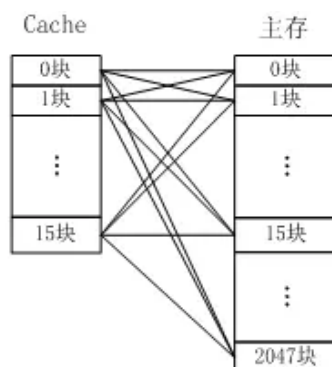
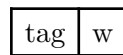


图 3-15 全相联映射示意图

不需要映射函数, 可以加载到任意一行

主存地址结构



只需要使用 tag 标识哪一个 block 来的就可以 w 为 $\log_2 W$, tag 至少有 $\log_2 B$ 个

优点: 放置 block 的位置完全自由

缺点: 需要实现复杂电路用于查询

12.6 组相连 Set Associative Mapping

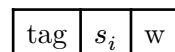
n-way 组相连指的是每一个组(set)中有 n 个 cache line 即

$$L = n \times s$$

通过映射函数映射到组中,然后组内是全相连 即

$$s_i = i \% s$$

$$\text{tag} = \frac{i}{s}$$



那么 j 有 $\log_2 s$ 位, tag 至少有 $\log_2 \frac{B}{s}$, w 为 $\log_2 W$

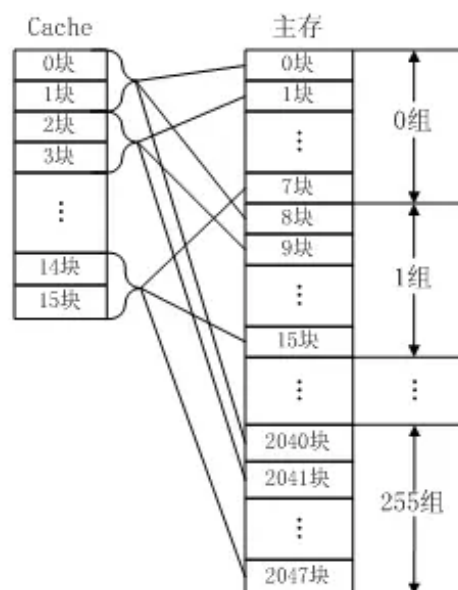


图 3-16 组相联映射示意图

优点:

1. 相比 direct mapping, 每一个主存块有多个可选的 cache 位置, 解决了相容性(contention)问题
2. 相比全相连, 其需要搜索的位置更少, 减少了硬件花费

缺点:

1. 虽然需要搜索的位置更少, 但是仍然需要 k 个比较器并行比较
2. 虽然其硬件成本比全相连低, 但是仍然比直接映射高

12.7 valid bit

就算 tag 相等也需要校验位, 因为刚刚通电时后没有任何数据

刚刚通电 valid bit 是 0,表示无效,然后载入数据后,校验位变为 1,表示有效

12.8 Replacement Algorithms

1. Direct Mapped Cache

不用策略,没有 hit 就替换

2. 组相连或者全相连

注意这个 cache 的实现是硬件上的(速度),LRU,FIFO,Random

12.9 写策略

如果 write 在缓存中,即 write hit

1. Write through

同时写主存和缓存

2. Write back

在缓存上修改,并使用 dirty bit 标记,等到从内存中被删除的时候再写回

如果 write miss

1. Write through

直接写入主存

2. Write back

先载入缓存,在缓存上修改,并使用 dirty bit 标记,等到从内存中被删除的时候再写回

	Write through	Write back
优点	内存中和缓存中一致	1. 速度更快 2. 一次访问主存可以写多个字
缺点	1. 所有写需要访问内存,占用总线 2. 整个系统变慢,因为要等待写完成才能读	1. 有一些主存的位置是无效的,只能通过 cache 操作 I/O 2. 需要校验位,占用额外的 cache 空间

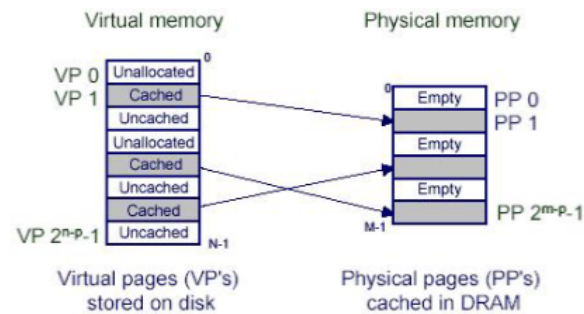
12.10 多级缓存

多个缓存构成存储结构,而且分为指令缓存和数据缓存 优点是:访问更加集中,局部性更强. 而且由于分开后更小,导致访问更加快速.

多级缓存系统算期望时间时候需要在一级没有命中时候考虑二级的期望.

13 虚存

虚存不是物理设备,而是一种抽象概念,目标是让每一个存储都觉得有足够大的空间,位于主存和硬盘之间



MMU:将物理地址转化为逻辑地址的硬件,通过查 page table(OS 管理)的方式

13.1 page

硬盘中称为 page,是一个 block 的连续字 主存中称为 page frame,是可以放下一个 page 的区域 用全相连映射 虚拟内存

虚页号 VPN	偏移 Offset
---------	-----------

物理内存

实页号 PPN	偏移 Offset
---------	-----------

页表记录了从物理内映射到逻辑内存的映射方式(存储在主存中),以数组形式存储,其中下标是 VPN,值是 PPN, PTBR(page table base register)指向正在当前程序页表的起始位置.

index	control bit	PP
0		PPN ₀
1		PPN ₁
2		disk
3		PPN ₃

注意从逻辑上来说不存储 index,就是按照虚页号从小到大存储的

Page Fault 缺页并不是错误,只是因为页不在主存中,类似于 cache miss

13.2 TLB(Translation Look-aside Buffer)转换检测缓冲区

首先一次对虚存访问的访问会占用 2 次内存操作

- 从内存中请求页表并获得物理地址
- 从主存中将数据返回

增加一个缓存,把一部分页表放入缓存之中,这样可以减少一次访问主存

VPN	PPN	control bit
-----	-----	-------------

其替换算法可以软件实现,且没有 write through 策略(只存储对应关系)

13.3 缺点

Internal fragmentation problem : 可能存在内存碎片,最后一个 page 可能不满

VPN	control bit	PPN
-----	-------------	-----

13.4 选取 page 的大小

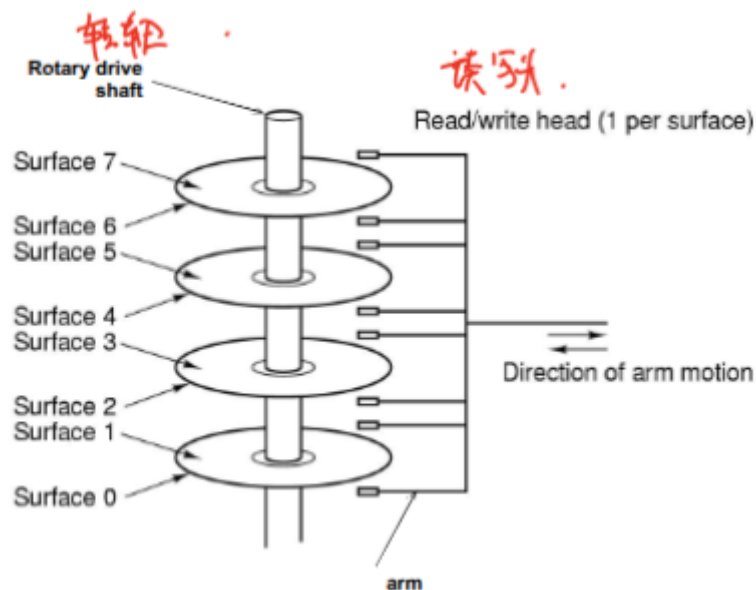
page 过大

优点

1. 页表小,占有主存小
2. 从辅存搬运数据更加高效(寻道时间短)

缺点 内存碎片大,浪费内存更多

14 磁盘 Magnetic Hard Disk



14.1 组织形式

surface: 盘面 track: 磁道,盘面划分成多个同心圆 Sector: 一个磁道有多个扇区,磁道切成若干段 cylinder: 柱面,在同一柱面不需要移动磁头臂,只需要移动磁头,也就是说同柱面访问要快

磁盘地址表示为(surface number, track number, sector number)

14.2 容量计算

容量 = surface 数目 × track 数目 × sector 数目 × 每个 sector 的字节数目

如果是给出盘片 platters,那么需要考虑是单面的还是双面的,一般默认双面,再乘上一个盘片上盘面的数目

14.3 延迟分析

主要延迟在于寻道,总延迟为寻道时间(找到正确的 track)加上旋转时间(等数据转到磁头下面)

15 指令集

15.1 两种指令形式

Register transfer:

$$R2 \leftarrow [LOC]$$

使用[]表示寄存器的内容,使用←表示向目标转移

Assembly-Language Notation:

Load R2, LOC

使用指令表示

15.2 RISC 和 CISC

15.2.1 RISC

内存

- 简单的地址模式
- 算数和逻辑操作在寄存器上
- 使用 load 和 store 命令传输数据

指令

- 所有指令都是一字长
- 指令集中指令少
- 每个程序执行更多指令

15.2.2 CISC

内存

- 复杂的地址模式
- 算数和逻辑操作在内存上
- 内存到内存数据传输

指令

- 指令可能比一个字长
- 更多的指令在指令集中
- 每个程序执行更少指令

15.3 顺序执行和 branching

取指令,执行指令,假定指令长度是为 4

$$[PC] = i$$

取指令后

$$[PC] = i + 4$$

branch 的时候取完指令应该 PC 指向顺序下一条,然后被执行时候赋值到跳转目标

15.4 opcode 的设计

假设一个指令集每一条指令的长度为16bit,每一个操作数的地址为6bit,分别为 0 地址,1 地址,2 地址指令,设计指令集的 opcode. 从 opcode 最短的指令开始设计,也就是 2 地址指令

idx	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
start	0	0	0	0	op1						op2					
end	1	1	1	0	op1						op2					

第 3 位留为 0 的目的是用于区分 2 地址指令和 1 地址指令,即 2 指令地址第 4 位为 0,而 1 指令地址第 4 位为 1. 其指令数为 $2^4 - 1 = 15$

然后设计一指令地址

idx	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
start	1	1	1	1	0	0	0	0	0	0	op1					
end	1	1	1	1	1	1	1	1	1	0	op1					

也就是说在3位为1的前提下,4 ~ 9位为000000 ~ 111110表示 1 地址指令. 共有 $2^6 - 1 = 63$

idx	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
start	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0
end	1	1	1	1	1	1	1	1	1	0	1	1	1	1	1	1

最后 0 地址不用留下一位,共有 $2^6 = 64$.

综上总共需要有 $15 + 63 + 64 = 142$ 种类 opcode.

15.5 寻址方式

- ① Immediate mode 带 # 表示是给出的是立即数
- ② Absolute mode 直接给出地址
- ③ Register mode 直接访问寄存器
- ④ Register Indirect mode 通过寄存器中的地址访问内存
- ⑤ Indirect mode 给出存储目标内存地址的内存地址
- ⑥ Indexed mode 给出一个寄存器和一个相对寄存器中地址的偏移量
- ⑦ Relative mode Indexed mode 使用 PC 的时候是 relative mode

RISC-type addressing modes.		
Name	Assembler syntax	Addressing function
Immediate	#Value	Operand = Value
Register	Ri	EA = Ri
Absolute	LOC	EA = LOC
Register indirect	(Ri)	EA = [Ri]
Index	X(Ri)	EA = [Ri] + X
Base with index	(Ri,Rj)	EA = [Ri] + [Rj]
EA = effective address		
Value = a signed number		
X = index value		

16 基本处理单元

16.1 多阶段

把操作分成多个简单阶段,每一个阶段中间使用寄存器保存状态,成为 multi-stage,在一个时钟周期中执行一个阶段

16.2 指令执行五步

1. 取指令,并增加 PC

2. 译码,并从 register file 中读取寄存器
3. 执行 ALU 运算
4. 如果需要读写内存,则在此步读写内存
5. 如果需要将结果写入目标寄存器,此步写入目标寄存器

16.3 硬件执行五步

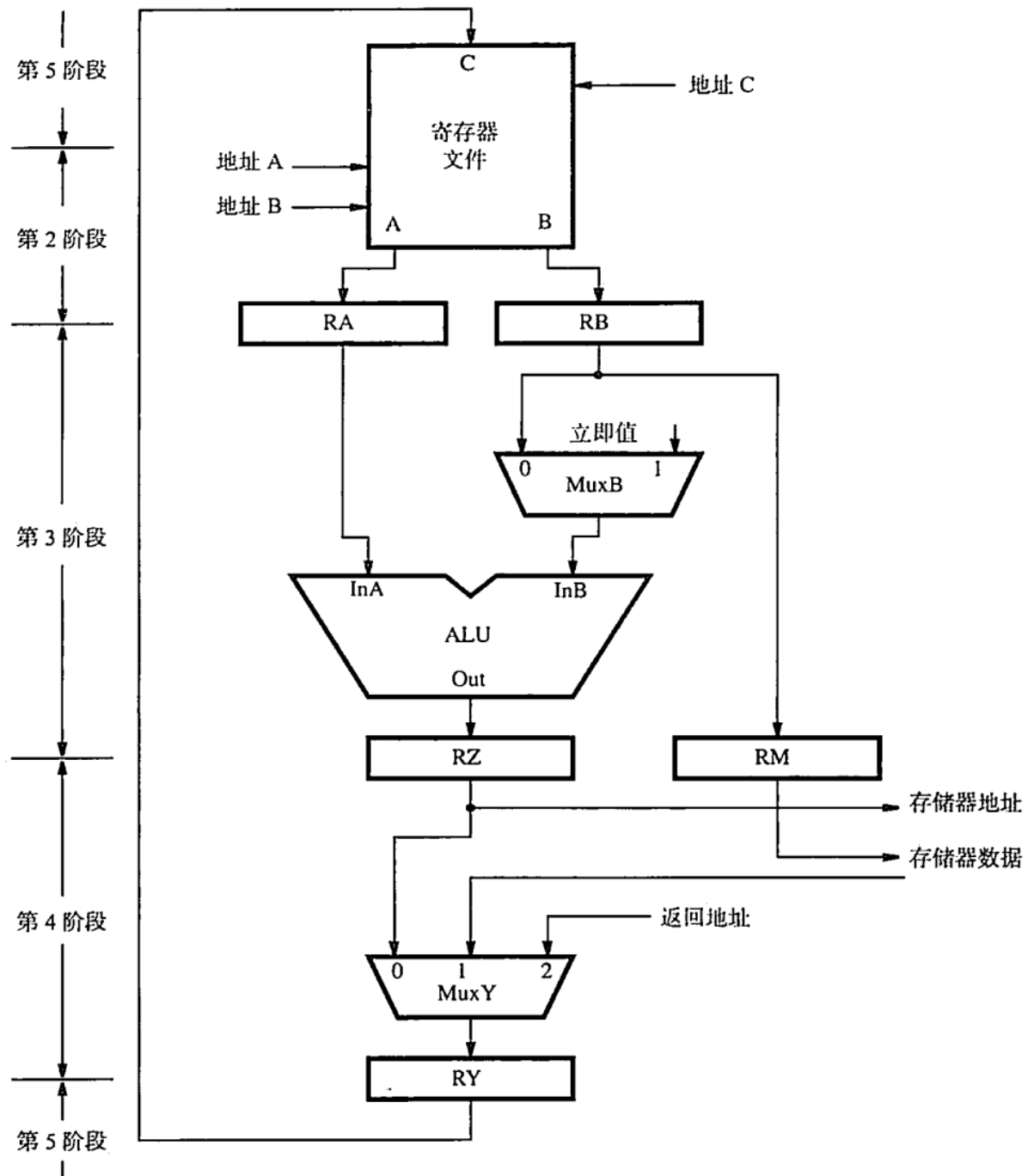


图 5-8 处理器中的数据通路

ADD R3,R4,R5 的五阶段

1. $\text{Memory Address} \leftarrow [\text{PC}], \text{Read Memory}, \text{IR} \leftarrow \text{Memory Data}, \text{PC} \leftarrow [\text{PC}] + 4$
2. Decode Instruction, $\text{RA} \leftarrow [\text{R4}], \text{RB} \leftarrow [\text{R5}]$
3. $\text{RZ} \leftarrow [\text{RA}] + [\text{RB}]$

4. $RY \leftarrow [RZ]$
5. $R3 \leftarrow [RY]$

LOAD R5, X(R7) 的五阶段

1. Memory Address $\leftarrow [PC]$, Read Memory, $IR \leftarrow$ Memory Data, $PC \leftarrow [PC]+4$
2. Decode Instruction, $RA \leftarrow [R7]$
3. $RZ \leftarrow [RA]+$ immediate value X
4. Memory Address $\leftarrow [RZ]$, Read Memory, $[RY] \leftarrow$ Memory Data
5. $R5 \leftarrow [RY]$

Store R6, X(R8)的五阶段

1. Memory Address $\leftarrow [PC]$, Read Memory, $IR \leftarrow$ Memory Data, $PC \leftarrow [PC]+4$
2. Decode Instruction, $RA \leftarrow [R8], RB \leftarrow [R6]$
3. $RZ \leftarrow [RA]+$ immediate value X, $RM \leftarrow [RB]$
4. Memory Address $\leftarrow [RZ]$, Memory Data $\leftarrow [RM]$, Write Memory
5. No action

Unconditional Branch 的五阶段

1. Memory Address $\leftarrow [PC]$, Read Memory, $IR \leftarrow$ Memory Data, $PC \leftarrow [PC]+4$
2. Decode Instruction
3. $PC \leftarrow [PC] +$ Branch offset
4. No action
5. No action

Conditional Branch: Branch_if_[R5]=[R6] LOOP

1. Memory Address $\leftarrow [PC]$, Read Memory, $IR \leftarrow$ Memory Data, $PC \leftarrow [PC]+4$
2. Decode Instruction, $RA \leftarrow [R5], RB \leftarrow [R6]$
3. Compare $[RA]$ to $[RB]$, if $[RA]==[RB]$ then $PC \leftarrow [PC] +$ Branch offset
4. No action
5. No action

Call_register R9

1. Memory Address $\leftarrow [PC]$, Read Memory, $IR \leftarrow$ Memory Data, $PC \leftarrow [PC]+4$
2. Decode Instruction, $RA \leftarrow [R9]$
3. $PC-TEMP \leftarrow PC, PC \leftarrow [RA]$
4. $RY \leftarrow [PC-TEMP]$
5. Register LINK $\leftarrow [PY]$

Return

1. Memory Address $\leftarrow [PC]$, Read Memory, $IR \leftarrow$ Memory Data, $PC \leftarrow [PC]+4$
2. Decode Instruction, $RA \leftarrow [LINK]$
3. $PC \leftarrow [RA]$
4. No action
5. No action

16.4 控制信号

基于图片,写控制信号,注意多位控制信号写成二进制

MFC,如果没有在缓存 hit 上,那么就会导致需要前往主存查找,导致花费不止一个时钟周期,为此设计 MFC 信号,等到接受到 MFC 信号后再读出新的 IR

16.5 Generation of Control Signals

16.5.1 Hardwired Control

输入是解码后的 IR,step 控制,外部输入型号,ALU 运算结果条件码

16.5.2 Microprogramming

基本概念,微指令(microinstruction)构成微程序(Microprogramme),存放在控制存储器(Control Store) 水平型: 一位表示一个控制信号

垂直型:把不冲突的控制信号放到分在一组,然后译码

16.5.3 优缺点比较

	Hardwired Control	Microprogramming
优点	速度快	1. 成本低 2. 灵活
缺点	1. 成本高 2. 不灵活 3. 难设计	速度慢

17 I/O 系统

17.1 访问 I/O 设备

访问 I/O 设备也是通过地址的方式实现.有两种编址方式

1. Memory-mapped I/O 把其映射到内存地址
2. Separated I/O 把其单独映射到一个地址

比较

	Memory-mapped I/O	Separated I/O
优点	只要能够访问内存就能够访问 I/O 设备	处理更少的地址线
缺点	浪费了宝贵的寻址空间	需要特殊的访问指令

17.2 I/O Device Interface

I/O 的速度比 CPU 和 RAM 慢,不同设备有各种不同,所以需要在 I/O 和内部网络之间放一个接口,用于提供数据传输方法和状态以及控制信号的交换. 包括

- 数据
- 状态
- 控制寄存器

17.3 Program-controlled I/O(polling)

让 CPU 进行轮询和数据搬运 例子是 CPU 不断查询 KIN,当发现 KIN 为 1 的时候就从 KBD_DATA 中读出数据,并将 KIN 设置为 0

17.4 interrupt

PSW 是程序状态位(Program word status).

由 I/O 设备产生中断(IR)信号,CPU 接收到中返回一个 Acknowledge 表示已经接收到中断了,然后 CPU 开始执行 ISR(响应中断请求),在发出中断到开始执行这一段时间称为中断延迟

	subroutine	interrupt-service routine
不同	子程序由父程序调用,完成父程序要求的任务,对状态信息和寄存器内容的变化是可预期的	中断服务程序可能与当前执行的程序无关,进入前需保存可能被意外修改的状态信息和寄存器内容,返回前恢复

中断的响应,执行完成当前正在执行的那一条指令,然后挂起正在执行的程序,保存 PC,Registers 在 PCB(Process Control Block)中,开始执行中断响应程序.在执行完成后需要恢复原来的条件状态位(condition code flags)和 PC,这个过程可以由硬件解决,也可以由软件解决.现在硬件一般只保存最少信息(PC 和 PSW)其他信息需要在 ISR 开始时候保存在 ISR 结束时候恢复.

17.4.1 中断的使能和禁止

在 PSW 中有 Interrupt-enable bit,如果为 1 就会才会响应 interrupt,如果是 0 会忽略.

在 I/O 设备接口上有 interrupt-enable bit ,作用是让 I/O 设备只有在 CPU 允许的时候才能生成中断

17.4.2 中断源的识别

1. 多根中断请求线,容易识别,但是即使如此也可能有一根中断线连接多个 I/O 设备的情况
2. 一根中断线,使用
 - polling:轮询,处理器依次查询每一个接口的 IRQ
 - Vectored Interrupt:返回一个 Vector code,然后查询 Interrupt vector table(vector code 为 index,值为对应 I/O 设备的 ISR 地址),然后将查询结果赋值给 PC

17.4.3 多级中断(嵌套中断)

给处理器一个优先级,只收取优先级比自己高的中断信号,当受到比自己优先级高的中断就转而处理他,并把处理器的优先级设为更高的中断.

1. 每一个设备有单独的中断和响应线: 优先级仲裁电路
2. 多设备共享
 - polling:轮询优先级
 - Daisy Chain: 优先级高的离处理器近,最先截获信号的优先级最高
 - Priority Group: 设备按照组划分,先响应优先级高的组

17.5 DMA

Direct Memory Access 用于直接在外设设备和主存之间传输数据,DMAC(Direct Memory Access Control)被用于从 CPU 接管系统,在传输时候 CPU 不参与.

1. CPU 准备 DMA 操作,将信息传输给 DMAC
2. 当设备准备好传输后,DMAC 接受总线控制,CPU 继续其他工作
3. 在完成后 DMAC 发出一个中断请求



DMAC 的作用

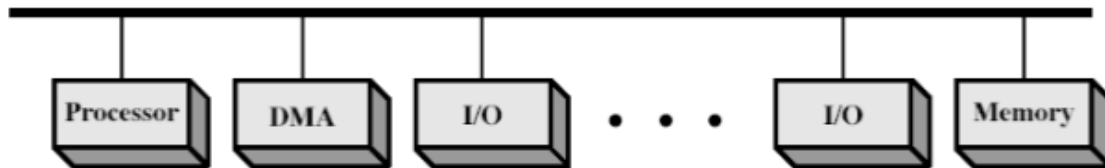
1. 提供主存地址
2. 提供总线控制信号
3. 自增连续字
4. 跟踪数据传输量

三个模式

- Burst mode: 一旦被授权就传输所有数据,用于加载程序或数据文件,缺点是 CPU 利用率低,处理器长时间不活跃
- Cycle Stealing mode 周期窃取模式 一次只发送一个字
- Transparent Mode: 只在 cpu 不使用总线的时候发送,优点是 CPU 不停止,缺点是硬件难以设计

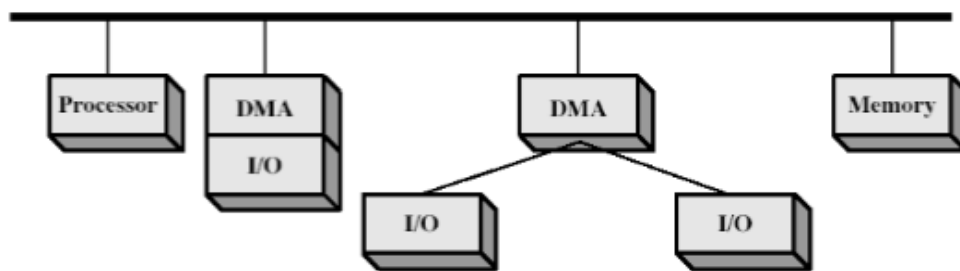
DMA 配置

1. 单总线、分离式 DMA 控制器: CPU 两次挂起(IO → DMAC 一次,DMAC → memory 一次)



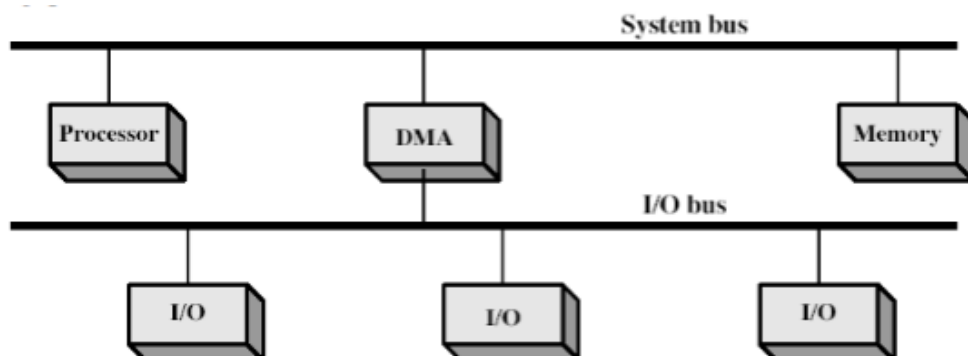
(a) Single-bus, detached DMA

2. 单总线,集成 DMA 控制: CPU 挂起一次(DMAC → memory 一次)



(b) Single-bus, Integrated DMA-I/O

3. 单独 I/O 总线:CPU 挂起一次(DMAC → memory 一次)



17.6 三种方法比较

	Polling	Interrupt	DMA
优点	实现简单,适用于外部源少的系统	不需要 CPU 反复检查,解决了忙等问题,IO 设备主动中断	传输过程 CPU 不参与,高速块传输
缺点	CPU 频繁 cheking(waiting loop),不进行有效计算(忙等)	但是传输数据仍然需要 CPU 介入,且速率受到限制,需要大量等待时间	突发模式使得 CPU 长期不活跃,透明模式电路难以设计